



Instituto Politécnico Nacional

Escuela Superior de Computo



Materia: Aplicaciones para Comunicaciones en Red

Profesor: Tanibet Pérez De los Santos Mondragón

Integrantes: Gil Valentín Oswaldo

Grupo: 6CV1 **Fecha:** 27/04/2026

Actividad: Ejercicio 3.3 Event

Instrucciones:

Implementa un sistema cliente-servidor en Python utilizando threading.Event para simular el final de una partida cuando un jugador pierde.

Requisitos:

1. Servidor:

- Mantiene la partida en ejecución.

```
PS D:\Documentos de Escritorio\Escuela\Aplicaciones para comunicaciones en Red\Unidad 3\Ejercicio 3.3 Event> python Clie
nte.py
Buscando partida...
-> Conectado. Soy el Jugador 1
Jugador 1 sigue jugando...
Jugador 1 sigue jugando...
Jugador 1 sigue jugando...
Jugador 1 sigue jugando...
Jugador 1 sigue jugando...
Jugador 1 sigue jugando...
Jugador 1 sigue jugando...
Jugador 1 sigue jugando...
Jugador 1 sigue jugando...
Jugador 1 sigue jugando...
Jugador 1 sigue jugando...
Jugador 1 sigue jugando...
Jugador 1 sigue jugando...
Jugador 1 sigue jugando...
Jugador 1 sigue jugando...
Jugador 1 sigue jugando...
Jugador 1 sigue jugando...
Jugador 1 sigue jugando...
```

- Después de un tiempo aleatorio, selecciona (o simula) que un jugador ha perdido.

```

PS D:\Documentos de Escritorio\Escuela\Aplicaciones para comunicaciones en Red\Unidad 3\Ejercicio 3.3 Event> python Cliente.py
Buscando partida...
-> Conectado. Soy el Jugador 4
Jugador 4 sigue jugando...
Jugador 4 sigue jugando...
Jugador 4 sigue jugando...
Jugador 4 sigue jugando...
Jugador 4 sigue jugando...
Jugador 4 sigue jugando...
Jugador 4 sigue jugando...
Jugador 4 sigue jugando...
Jugador 4 sigue jugando...

[ALERTA DE RED] El servidor indica que el Jugador 1 perdio!
Jugador 4 ha sido notificado del fin de la partida.

```

- Activa un Event que representa el fin de la partida.

```

#Se selecciona al perdedor
perdedor = random.randint(1, NUM_JUGADORES)
print("\n" + "="*55)
print(f"[SERVIDOR] El jugador {perdedor} ha perdido. Fin de la partida.")
print("="*55 + "\n")

```

Para activar el evento primero tenemos que seleccionar un jugador que perdió de forma aleatoria.

```

for conn in clientes:
    try:
        #Enviamos el mensaje de FIN y el ID del perdedor
        conn.sendall(f"FIN:{perdedor}\n".encode('utf-8'))
        conn.close()
    except Exception:
        pass

```

Después de que seleccionamos al perdedor enviamos el mensaje a todos los jugadores el mensaje del jugador que perdió.

- Muestra en consola:
"El jugador X ha perdido. Fin de la partida."

```

Jugador 1 sigue jugando...
Jugador 1 sigue jugando...
Jugador 1 sigue jugando...

[ALERTA DE RED] El servidor indica que el Jugador 1 perdio!
Jugador 1 ha sido notificado del fin de la partida.

```

2. Clientes (jugadores):

- Cada cliente representa un jugador (usando hilos).

```
Buscando partida...  
-> Conectado. Soy el Jugador 1
```

```
Buscando partida...  
-> Conectado. Soy el Jugador 2
```

```
Buscando partida...  
-> Conectado. Soy el Jugador 3
```

```
Buscando partida...  
-> Conectado. Soy el Jugador 4
```

Cuando un jugador entra se le asigna una ID de jugador y empiezan a escuchar al servidor.

- Mientras la partida está activa, deben ejecutar un bucle mostrando algo como:
"Jugador X sigue jugando..." (puedes usar sleep).

```
# --- CICLO DEL JUEGO ---  
#Mientras el Hilo Escucha no encienda el Evento, seguimos jugando  
while not evento_fin_partida.is_set():  
    print(f"Jugador {mi_id_jugador} sigue jugando...")  
    time.sleep(random.uniform(0.5, 1.5))
```

```
Buscando partida...  
-> Conectado. Soy el Jugador 1  
Jugador 1 sigue jugando...  
Jugador 1 sigue jugando...  
Jugador 1 sigue jugando...  
Jugador 1 sigue jugando...  
Jugador 1 sigue jugando...  
Jugador 1 sigue jugando...  
Jugador 1 sigue jugando...  
Jugador 1 sigue jugando...  
Jugador 1 sigue jugando...
```

- Todos los clientes deben estar atentos al Event.

```

def escuchar_servidor(sock):
    global mi_id_jugador
    try:
        while True:
            data = sock.recv(1024).decode('utf-8')
            if not data:
                break

            mensajes = data.strip().split('\n')
            for msg in mensajes:
                if msg.startswith("ID:"):
                    mi_id_jugador = msg.split(":")[1]
                    print(f"Conectado. Soy el Jugador {mi_id_jugador}")

                elif msg.startswith("FIN:"):
                    perdedor = msg.split(":")[1]
                    print(f"\n[ALERTA DE RED] El servidor indica que el Jugador {perdedor} perdio!")

                    # --- ACTIVAMOS EL EVENTO ---
                    #Esto romperá el ciclo 'while' del hilo principal al instante
                    evento_fin_partida.set()
                    return

    except Exception:
        evento_fin_partida.set()

```

El Servidor avisa: El servidor envía un mensaje de texto que dice algo como "FIN:3\n" (indicando que perdió el jugador 3).

El Cliente escucha: Tu hilo secundario (la función `escuchar_servidor`) recibe los datos. La condición `elif msg.startswith("FIN:")` detecta esa palabra clave enviada por el servidor.

Se enciende el Evento: Al ejecutar `evento_fin_partida.set()`, el programa cambia el estado interno de ese Evento de `False` a `True`. Es literalmente presionar el interruptor de apagado general.

El Hilo principal reacciona: El hilo del jugador está atrapado en un ciclo `while not evento_fin_partida.is_set():`. En cuanto se ejecuta, el `is_set()` del hilo principal se vuelve verdadero, rompiendo el ciclo `while` y terminando el juego.

- Cuando el evento se activa, deben detener su ejecución y mostrar:

"Jugador X ha sido notificado del fin de la partida".

```
Buscando partida...
-> Conectado. Soy el Jugador 4
Jugador 4 sigue jugando...
Jugador 4 sigue jugando...
Jugador 4 sigue jugando...
Jugador 4 sigue jugando...
Jugador 4 sigue jugando...
Jugador 4 sigue jugando...
Jugador 4 sigue jugando...
Jugador 4 sigue jugando...

[ALERTA DE RED] El servidor indica que el Jugador 1 perdio!
Jugador 4 ha sido notificado del fin de la partida.
```

3. Sincronización:

- Usa `event.is_set()` o `event.wait()` para que los jugadores detecten cuándo termina la partida (cuidado con espera activa).

```
# Esperamos un instante a que el servidor nos asigne nuestro ID
while mi_id_jugador is None and not evento_fin_partida.is_set():
    time.sleep(0.1)

# --- CICLO DEL JUEGO ---
#Mientras el Hilo Escucha no encienda el Evento, seguimos jugando
while not evento_fin_partida.is_set():
    print(f"Jugador {mi_id_jugador} sigue jugando...")
    time.sleep(random.uniform(0.5, 1.5))
```

- Usa `event.set()` desde el servidor para notificar el evento global.

```
# --- ACTIVAMOS EL EVENTO ---
#Esto romperá el ciclo 'while' del hilo principal al instante
evento_fin_partida.set()
return
```